Actividad 1: Transformación de imágenes

Luis Fernando Salazar Hernández – A01738527

Instituto Tecnológico y de Estudios Superiores de Monterrey

Diseño de sistemas embebidos avanzados

Emmanuel Torres Ríos

14 de octubre de 2026

1. Descripción de la actividad.....3
2. Metodología y Decisiones de Implementación.....3
2.1. Diseño de la solución.....3
2.2. Entradas y salidas.....5
2.3. Condiciones de operación.....5
2.4. Consideraciones de implementación.....6
3. Evidencia de Ejecución y Resultados.....7
3.1. Imágenes obtenidas.....7
3.2. Verificación y calibración.....9
3.3. Errores en la medición.....9
3.4. Condiciones del experimento.....10
4. Conclusiones y Reflexión.....10
4.1. Elementos importantes para obtener los resultados.....10
4.2. Cómo modificar la respuesta.....11
5. Anexos y Código Fuente.....11

1. Descripción de la actividad

En esta actividad desarrollé una biblioteca en lenguaje C para aplicar transformaciones a imágenes en formato BMP de 24 bits. La biblioteca está en un solo archivo de cabecera (`img_lib.h`) y la pruebo con un programa principal (`main.c`) que aplica las seis transformaciones solicitadas sobre la misma imagen de entrada:

- `inv_hor_color`: inversión horizontal (espejo) a color.
- `inv_hor_bn`: inversión horizontal en escala de grises.
- `inv_ver_color`: inversión vertical a color.
- `inv_ver_bn`: inversión vertical en escala de grises.
- `blur_color`: desenfoque a color a partir de un kernel definido.
- `blur_bn`: desenfoque en escala de grises a partir de un kernel definido.

Para el desarrollo usé el lenguaje C con el compilador gcc y Visual Studio Code como editor. Como guía para la forma de leer y escribir el archivo usé el ejemplo de desenfoque visto en clase. La biblioteca solo depende de tres bibliotecas estándar: `stdio.h` para abrir, leer y escribir archivos (`fopen`, `fgetc`, `fputc`, `fread`, `fwrite`), `stdlib.h` para reservar y liberar memoria (`malloc` y `free`) y `string.h` para armar la ruta de salida (`strcat`). Como imagen de prueba usé `sample.bmp`, de 858 × 1080 píxeles.

Un archivo BMP de 24 bits tiene dos partes: una cabecera de 54 bytes con la información de la imagen (tamaño, ancho, alto y bits por píxel) y los datos de los píxeles. Cada píxel ocupa 3 bytes guardados en orden B, G, R (azul, verde, rojo), los números de varios bytes se guardan en formato little-endian (el byte menos significativo primero) y cada fila se completa con bytes de relleno hasta medir un múltiplo de 4 bytes.

Para asegurar que los resultados fueran correctos no bastaba con que las imágenes se vieran bien. Los elementos que garantizan el resultado son: la lectura exacta de la cabecera del archivo (ancho, alto y bits por píxel), el recorrido de los píxeles con ciclos de tamaño conocido, el manejo de los bytes de relleno que el formato BMP agrega al final de cada fila, el registro de localidades leídas y escritas en `output_log.txt` y la comparación de cada imagen de salida con el resultado esperado.

2. Metodología y Decisiones de Implementación

2.1. Diseño de la solución

Organicé la biblioteca para que las seis funciones siguieran la misma secuencia de pasos, de modo que cada una solo cambia en la parte de la transformación:

1. Armar la ruta de salida `./img/<nombre>.bmp` y abrir los archivos de entrada y salida.
2. Leer la cabecera de 54 bytes, copiarla sin cambios a la imagen de salida y obtener el ancho, el alto y los bytes de relleno.
3. Reservar memoria y leer los píxeles.
4. Aplicar la transformación y escribir la imagen resultante.
5. Registrar la operación en `output_log.txt`, liberar la memoria y cerrar los archivos.

En las cuatro inversiones los píxeles se guardan en tres arreglos, uno por canal (azul, verde y rojo), y la transformación consiste únicamente en cambiar el orden en que se escriben. En la inversión horizontal cada fila se escribe de la última columna a la primera. En la inversión vertical, de acuerdo con la definición indicada en clase, se escribe la imagen completa del último píxel al primero, lo que equivale a girarla 180°.

En los desenfoques cada píxel se reemplaza por el promedio de sus vecinos dentro de una ventana del tamaño del kernel. La imagen se lee por filas completas y el promedio se hace en dos pasadas, primero horizontal y después vertical, lo que reduce el número de sumas por píxel de kernel² a 2 × kernel (con kernel 15, de 225 a 30); es decir, el trabajo pasa de crecer con el cuadrado del kernel a crecer de forma lineal. Esta es una comparación del número de operaciones, no una medición de tiempo. El radio de la ventana se calcula como k = kernel / 2 con división entera, así que con kernel 15 se toman 7 vecinos a cada lado del píxel. En los bordes solo se promedian los vecinos que existen, para que las orillas de la imagen no se oscurezcan. El Listado 1 muestra la pasada horizontal.

```
522  int k = kernel_size / 2;
523
524  // Paso intermedio: desenfoque horizontal
525  for (int y = 0; y < height; y++) {
526      for (int x = 0; x < width; x++) {
527          int sum = 0, count = 0;
528
529          for (int dx = -k; dx <= k; dx++) {
530              int nx = x + dx;
531              if (nx >= 0 && nx < width) {
532                  sum += input_rows[y][nx * 3];
533                  count++;
534              }
535          }
536
537          int index = x * 3;
538          temp_rows[y][index + 0] = sum / count;
539          temp_rows[y][index + 1] = sum / count;
540          temp_rows[y][index + 2] = sum / count;
541      }
542
543      // Copiar padding sin modificar
544      for (int p = width * 3; p < row_padded; p++) {
545          temp_rows[y][p] = input_rows[y][p];
546      }
547  }
```

Listado 1. Fragmento de img_lib.h (líneas 522–547): pasada horizontal del desenfoque en blur_bn.

Para las versiones en escala de grises usé la fórmula de luminosidad Gris = 0.21·R + 0.72·G + 0.07·B, que da más peso al verde porque el ojo humano es más sensible a ese color. La conversión se hace al momento de leer cada píxel, y el valor de gris se escribe tres veces (B, G y R iguales) para que el archivo siga siendo de 24 bits y coincida con la cabecera copiada.

2.2. Entradas y salidas

Cada función recibe la ruta de la imagen original y el nombre con el que se guardará el resultado:

Función	Entradas	Salidas

`inv_hor_color`
`inv_hor_bn`
`inv_ver_color`
`inv_ver_bn`
	`mask`: nombre de la imagen de salida.
	`path`: ruta de la imagen BMP original.
		Imagen `./img/<mask>.bmp` y una entrada en `output_log.txt`.

`blur_color` `blur_bn`	`input_path`: ruta de la imagen BMP original. `name_output`: nombre de la imagen de salida. `kernel_size`: tamaño del kernel (impar).	Imagen `./img/<name_output>.bmp` y una entrada en `output_log.txt`.
`read_header`	`path`: ruta de la imagen BMP.	Ancho, alto y bits por píxel impresos en pantalla.

2.3. Condiciones de operación

- Imágenes BMP de 24 bits, sin compresión y con cabecera de 54 bytes. Lo verifiqué con `read_header`, que para la imagen de prueba muestra ancho 858, alto 1080 y 24 bits por píxel.
- La carpeta `img` debe existir en el directorio desde donde se ejecuta el programa. Si no existe, la función muestra “Error abriendo archivos.” y no continúa.
- El kernel debe ser impar para que la ventana quede centrada en el píxel. En las pruebas usé un kernel de 15.
- Compilación con `gcc main.c -o img_lib` y ejecución desde la carpeta que contiene `sample.bmp`.

Un detalle importante de la imagen de prueba es su ancho. En BMP cada fila debe ocupar un múltiplo de 4 bytes. Con 858 píxeles por fila se tienen 858 × 3 = 2,574 bytes de color, que no es múltiplo de 4, así que el archivo agrega 2 bytes de relleno al final de cada fila (2,576 bytes por fila). Este detalle influye directamente en cómo se debe leer y escribir la imagen, como explico en la siguiente sección.

2.4. Consideraciones de implementación

Durante el desarrollo puse especial atención en la forma de leer y escribir el archivo, porque noté que pequeños detalles del formato pueden hacer que el programa lea datos que no existen, escriba fuera de la memoria reservada o interprete el relleno como si fueran píxeles. Lo delicado es que en esos casos el programa normalmente no muestra ningún error y la imagen incluso puede parecer correcta a simple vista. La siguiente tabla resume lo que hace el código en cada punto y por qué es importante:

Aspecto	Qué hace el código	Por qué es importante
Lectura de píxeles	Dos ciclos `for` que leen exactamente ancho × alto píxeles.	Si se leyera hasta detectar el fin del archivo, la última lectura devolvería EOF (255) y ese valor se guardaría fuera del arreglo.
Relleno de fila	Calcula el relleno con `(ancho*3 + 3) & ~3`; lo salta al leer y lo escribe en 0 al final de cada fila.	Con 858 px de ancho hay 2 bytes de relleno por fila. Si se leyeran como color, cada fila quedaría desfasada y la imagen saldría inclinada y con los colores cambiados.
Índices del recorrido inverso	Recorre cada fila de `ancho - 1` hasta `0`.	Los índices válidos de una fila van de 0 a ancho − 1. Empezar en `ancho` tomaría un píxel de la fila siguiente y, en la última fila, leería fuera del arreglo.
Lectura de la cabecera	Arma el ancho y el alto con sus 4 bytes (`xx[18]` a `xx[21]` y `xx[22]` a `xx[25]`).	La cabecera guarda cada dato en 4 bytes; así el valor corresponde completo al formato.
Apertura de archivos	Verifica que `fopen` haya abierto la entrada y la salida.	Si falta la carpeta `img` o la imagen, el programa avisa en lugar de fallar sin indicar la causa.
Registro	Escribe ancho × alto como localidades leídas y escritas.	Como los ciclos hacen exactamente ese número de accesos, el valor registrado es exacto.

El Listado 2 muestra cómo se lee la cabecera y se calcula el relleno, y el Listado 3 cómo se leen los píxeles con ciclos acotados, saltando el relleno al terminar cada fila:

```
69  long ancho, alto;
70  unsigned char xx[54];
71  for(int i=0; i<54; i++) {
72    xx[i] = fgetc(image);
73    fputc(xx[i], outputImage);   //Copia cabecera a nueva imagen
74  }
75  ancho = (long)xx[21]*16777216+(long)xx[20]*65536+(long)xx[19]*256+(long)xx[18];
76  alto = (long)xx[25]*16777216+(long)xx[24]*65536+(long)xx[23]*256+(long)xx[22];
77  int padding = ((ancho * 3 + 3) & (~3)) - ancho * 3;   //Bytes de relleno por fila
```

Listado 2. Fragmento de img_lib.h (líneas 69–77): lectura de la cabecera y cálculo del relleno en inv_hor_color.

```
83  // Lectura de pixeles
84  long j = 0;
85  for (int i = 0; i < alto; i++) {
86      for (int x = 0; x < ancho; x++) {
87          arr_in_b[j] = fgetc(image);
88          arr_in_g[j] = fgetc(image);
89          arr_in_r[j] = fgetc(image);
90          j++;
91      }
92      for (int p = 0; p < padding; p++) fgetc(image);   //Salta el relleno
93  }
```

Listado 3. Fragmento de img_lib.h (líneas 83–93): lectura de píxeles con ciclos acotados en inv_hor_color.

Para la escritura, el Listado 4 muestra el recorrido de la inversión horizontal: cada fila se escribe de la última columna a la primera y al final se agregan los bytes de relleno en 0, para que la fila de salida conserve el tamaño que exige el formato.

```
95  // Escribir pixeles invertidos
96  for (int i = 0; i < alto; i++) {
97      for (int y = ancho - 1; y >= 0; y--) {
98          long index = (i * ancho) + y;
99          fputc(arr_in_b[index], outputImage);
100         fputc(arr_in_g[index], outputImage);
101         fputc(arr_in_r[index], outputImage);
102     }
103     for (int p = 0; p < padding; p++) fputc(0, outputImage);
104 }
```

Listado 4. Fragmento de img_lib.h (líneas 95–104): escritura de la inversión horizontal con relleno al final de cada fila.

Mantuve el código lo más sencillo posible: no agregué validaciones para casos que no se presentan con las imágenes de la práctica, como cabeceras extendidas o imágenes con otra profundidad de color.

3. Evidencia de Ejecución y Resultados

3.1. Imágenes obtenidas

La Figura 1 muestra la imagen de entrada y la Figura 2 las seis imágenes que generó la biblioteca con un kernel de 15.



Figura 1. Imagen de entrada sample.bmp (858 × 1080 píxeles, 24 bits).

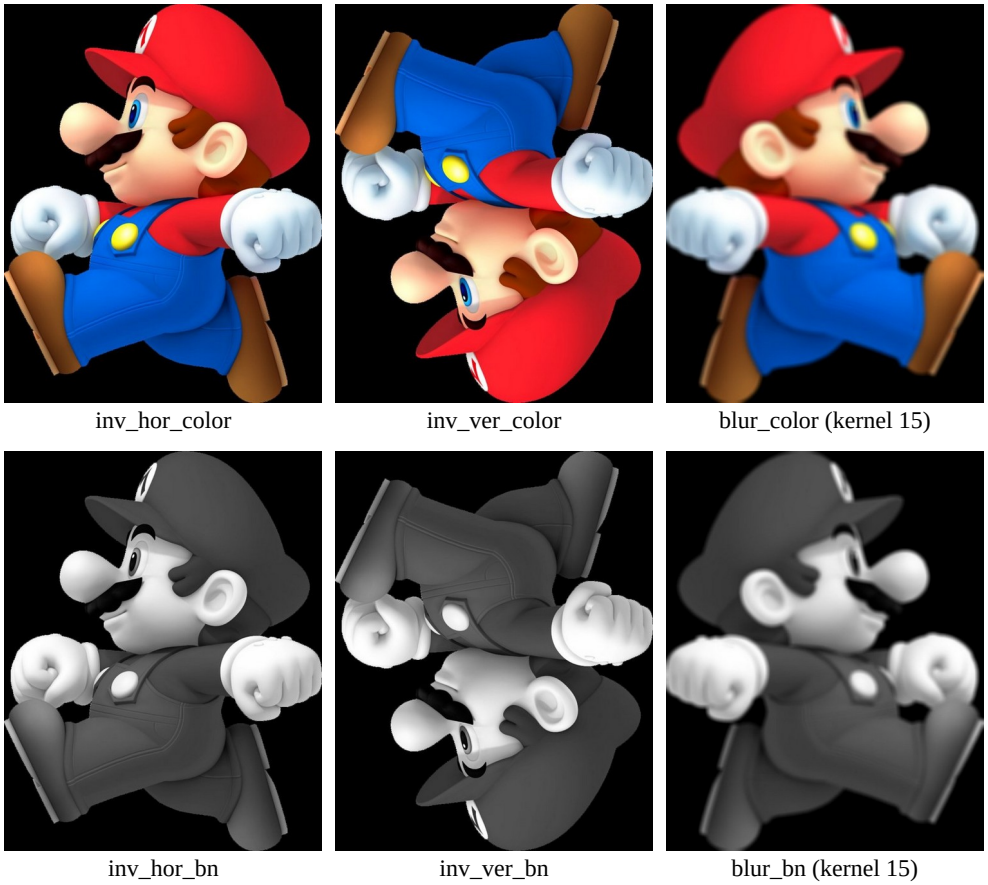


Figura 2. Imágenes generadas por las seis funciones de img_lib.h.

La comparación entre lo esperado y lo obtenido para cada par de funciones es la siguiente:

- Inversión horizontal (inv_hor_color e inv_hor_bn): se esperaba invertir cada fila sin cambiar el orden de las filas. Se obtuvo el personaje mirando hacia el lado contrario a la misma altura, como en un espejo, sin desfase en los bordes.
- Inversión vertical (inv_ver_color e inv_ver_bn): se esperaba que el último píxel pasara a ser el primero. Se obtuvo la imagen de cabeza y también espejeada, que equivale a girarla 180°.
- Desenfoque con kernel 15 (blur_color y blur_bn): se esperaba suavizar los contornos sin desplazar la imagen ni oscurecer las orillas. Se obtuvo un desenfoque uniforme en toda la imagen, incluidos los bordes.
- Versiones en escala de grises: se esperaba la misma geometría que en color, con un solo valor por píxel. Las tres conservan la geometría; colores distintos como el rojo de la gorra y el azul del overol quedan en tonos de gris parecidos, porque la fórmula pondera cada canal según su brillo.

Ninguna de las imágenes presenta inclinación, franjas ni colores cambiados, lo que confirma que los 2 bytes de relleno de cada fila se manejaron correctamente tanto al leer como al escribir.

3.2. Verificación y calibración

Para verificar los resultados usé como referencia la información de la cabecera, el registro de ejecución y la comparación de cada imagen de salida con el resultado esperado:

Prueba	Resultado esperado	Resultado obtenido
Medidas de la imagen con `read_header`	BMP de 24 bits	Ancho 858, alto 1080 y 24 bits por píxel
Registro en `output_log.txt` (6 funciones)	858 × 1080 = 926,640 localidades	926,640 leídas y 926,640 escritas en cada función
Inversión horizontal	Imagen reflejada, sin desfase en los bordes	Reflejo correcto (Figura 2)
Inversión vertical	Imagen girada 180°	Imagen de cabeza y espejeada (Figura 2)
Desenfoque	Contornos suavizados, sin desplazamiento ni bordes oscuros	Desenfoque uniforme en toda la imagen (Figura 2)
Manejo del relleno	Filas alineadas, sin desfase entre ellas	Ninguna imagen inclinada ni con colores cambiados

La cabecera sirve como referencia de calibración: el ancho y el alto que se leen de ella definen exactamente cuántos píxeles se procesan, y el registro confirma que cada función trabajó con 926,640 localidades. Por su parte, el relleno funciona como un indicador muy sensible: si se leyera aunque fuera un byte de más o de menos por fila, el error se acumularía fila tras fila y la imagen saldría inclinada y con los colores cambiados, algo que no ocurre en ninguna de las seis salidas.

3.3. Errores en la medición

- Truncamiento en la escala de grises: al guardar 0.21·R + 0.72·G + 0.07·B en un `unsigned char`, C elimina los decimales. El error máximo es de 1 nivel de gris (de 255), una diferencia que no es perceptible.
- División entera en el desenfoque: cada pasada divide la suma entre `count` con división entera, así que puede perder hasta 1 nivel por pasada.
- Acumuladores del desenfoque: las sumas se guardan en variables `int` de 32 bits y se dividen entre `count` antes de convertir a 8 bits. Con kernel 15 la suma máxima es 15 × 255 = 3,825, muy por debajo del límite de un `int`, así que no hay desbordamiento.

- Bordes: en las orillas la ventana tiene menos vecinos. Al dividir entre `count` el promedio sigue siendo correcto, pero el suavizado es menor que en el centro de la imagen.
- Alcance del registro: `output_log.txt` cuenta píxeles (localidades lógicas), no bytes; los bytes de relleno no se incluyen en el conteo.

3.4. Condiciones del experimento

- Imagen de entrada: `sample.bmp`, 858 × 1080 píxeles, 24 bits, 2,782,134 bytes y 2 bytes de relleno por fila.
- Kernel de desenfoque: 15, aproximadamente el 1.7 % del ancho de la imagen.
- Compilación con gcc y ejecución desde la carpeta del proyecto, con la carpeta `img` creada previamente.
- `output_log.txt` se abre en modo de agregar (`"a"`), por lo que lo borré antes de la corrida para registrar únicamente las seis funciones.

El tamaño del kernel controla la intensidad del desenfoque: con kernel 15 cada píxel se promedia con 7 vecinos a cada lado en cada pasada. Un kernel mayor promedia más vecinos y produce un desenfoque más marcado, y su costo por píxel crece de forma lineal gracias a las dos pasadas, en lugar de crecer con el cuadrado del kernel.

4. Conclusiones y Reflexión

4.1. Elementos importantes para obtener los resultados

La parte más importante de esta práctica no fue aplicar las transformaciones en sí, que son sencillas, sino leer y escribir el archivo exactamente como lo define el formato BMP. Por eso puse especial atención en tres puntos: recorrer los píxeles con ciclos de tamaño conocido, respetar los índices válidos de cada fila y tomar en cuenta el relleno.

El relleno resultó especialmente relevante con la imagen de prueba. Con 858 píxeles de ancho, cada fila lleva 2 bytes de relleno; si no se consideraran, cada fila se leería desfasada 2 bytes respecto a la anterior, al llegar a la última fila el desfase acumulado sería de 2,160 bytes (720 píxeles) y los canales de color terminarían intercambiados. Además, esos bytes extra harían que se lean más valores de los que caben en los arreglos. Lo mismo ocurre con un ciclo que lee una vez de más o con un índice que empieza una posición fuera de la fila: el programa puede terminar sin mostrar ningún error, así que el problema pasa desapercibido. En un sistema embebido esto es todavía más delicado, porque una escritura fuera de memoria puede corromper otros datos del programa sin dar aviso.

Gracias a estas consideraciones la biblioteca funciona con cualquier ancho de imagen, no solo con anchos múltiplos de 4, y en ningún momento lee datos inexistentes ni escribe fuera de la memoria reservada. Las seis imágenes de salida, sin desfase ni colores cambiados, y el registro con 926,640 localidades por función lo confirman.

4.2. Cómo modificar la respuesta

De acuerdo con los datos del experimento, la respuesta se puede ajustar de las siguientes formas:

- Tamaño del kernel: es el parámetro principal del desenfoque: valores pequeños suavizan poco y conservan el detalle, mientras que valores grandes producen un desenfoque más marcado. Conviene elegirlo en proporción al ancho de la imagen y siempre impar.
- Tipo de promedio: el promedio uniforme puede sustituirse por pesos gaussianos, que dan más importancia a los vecinos cercanos y producen un desenfoque más natural. El filtro seguiría siendo separable en dos pasadas.
- Definición de la inversión vertical: puede ajustarse para invertir únicamente el orden de las filas si se requiere un volteo de arriba hacia abajo sin el efecto espejo.
- Paralelización: la estructura elegida (un arreglo por canal en las inversiones y filas independientes en el desenfoque) facilita el siguiente paso del módulo, que es paralelizar estas funciones con OpenMP, ya que cada fila se puede procesar de forma independiente. La mejora real dependerá del tamaño de la imagen y del procesador, por lo que habrá que medir el tiempo de ejecución antes y después.

5. Anexos y Código Fuente

El código desarrollado para esta práctica, junto con la imagen de prueba, las imágenes resultantes y el registro de ejecución, se encuentra disponible para su revisión en mi repositorio de GitHub de la materia, dentro de la carpeta del Módulo 3:

github.com/Fer1307/te2004b-advanced-embedded-systems-design

La carpeta de la actividad contiene los siguientes archivos:

- `img_lib.h`: biblioteca con las funciones `read_header`, `inv_hor_color`, `inv_hor_bn`, `inv_ver_color`, `inv_ver_bn`, `blur_color` y `blur_bn`.
- `main.c`: programa de prueba que aplica las seis transformaciones a `sample.bmp` con kernel 15.
- `output_log.txt`: registro de localidades leídas y escritas por cada función.
- `img/`: carpeta con las seis imágenes resultantes.

Para compilar y ejecutar el programa:

```
mkdir img
gcc main.c -o img_lib
.\img_lib.exe
```

Primeras entradas de `output_log.txt` después de la ejecución:

```
Función: inv_hor_color, con sample.bmp
Localidades totales leídas: 926640
Localidades totales escritas: 926640
-------------------------------------
Función: inv_hor_bn, con sample.bmp
Localidades totales leídas: 926640
Localidades totales escritas: 926640
-------------------------------------
```